Dilation Morphological Operator

HPQC Exam Accelerated Computing

Student: Alberto Amaduzzi

May 2026

1 Introduction

The *dilation morphological operator* expands shapes in a grayscale image by replacing each pixel (i, j) with the minimum grey level found inside a circular stencil mask of radius `MASK_RADIUS` $= 5$ (diameter `MASK_WIDTH` $= 11$) centred on that pixel. The operation requires reading from one buffer and writing to another to ensure synchronous updates; repeated application is controlled by a command-line parameter `rep`.

This report describes a CUDA GPU implementation and an OpenMP CPU implementation, analyses their performance, and summarises the results obtained on the Leonardo HPC cluster (NVIDIA A100 GPU, `boost_usr_prod` partition).

2 Implementations

2.1 OpenMP (`omp-dilate.c`)

The OpenMP version is a minimal modification of the serial `dilate.c`: a single `#pragma omp parallel for collapse(2)` directive is added to the two outer loops over rows and columns of the `dilate()` function. Because each output pixel is written by exactly one thread and reads are from a separate input buffer, there are no data races and no synchronisation primitives are needed beyond the implicit barrier at the end of the parallel region.

Compilation:

```
gcc -std=c99 -O2 -fopenmp omp-dilate.c -o omp-dilate
```

2.2 CUDA (`cuda-dilate.cu`)

2.2.1 Problem decomposition

A two-dimensional grid of two-dimensional thread blocks is used. Each CUDA thread is responsible for exactly one output pixel, identified by

$$\texttt{col} = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x},$$
$$\texttt{row} = \texttt{blockIdx.y} \cdot \texttt{blockDim.y} + \texttt{threadIdx.y}.$$

The block size is fixed at $16 \times 16 = 256$ threads. Grid dimensions are computed as $\lceil W/16 \rceil \times \lceil H/16 \rceil$. Threads that fall outside the image bounds return immediately.

2.2.2 Constant memory for the mask

The stencil mask is a small ($11 \times 11 = 121$ bytes), read-only array that is identical for every thread. It is stored in CUDA *constant memory* (`__constant__ unsigned char d_mask[]`), which is cached and broadcast to all threads in a warp in a single transaction, avoiding repeated global-memory reads.

2.2.3 Ping-pong buffers

To support `rep` > 1 iterations entirely on the GPU (avoiding costly host $\leftrightarrow$ device transfers after each step), two device buffers `d_buf[0]` and `d_buf[1]` are allocated once. At each iteration the kernel reads from `d_buf[cur]` and writes to `d_buf[next]`; after the kernel completes the indices are swapped:

```
cur  = 1 - cur;
next = 1 - cur;
```

This results in exactly **two** PCIe transfers regardless of `rep`: one upload at the start and one download at the end.

2.2.4 Timing methodology

Each measurement is repeated **five** independent times and the average is reported, in line with the assignment recommendation to "level out statistical fluctuations". `cudaDeviceSynchronize()` is called before stopping the timer to ensure all GPU work has completed. Throughput is expressed in two complementary units:

- **Mpixels/s** — algorithmic throughput: $T = (W \times H \times \texttt{rep})/t_{\text{avg}}/10^6$
- **GB/s** — effective memory bandwidth (one read + one write per pixel per iteration): $B = 2 \times (W \times H \times \texttt{rep})/t_{\text{avg}}/10^9$

Compilation:

```
nvcc -O2 cuda-dilate.cu -o cuda-dilate
```

3 Performance Results

All benchmarks were run on a Leonardo `boost_usr_prod` node (1 NVIDIA A100 SXM4-40GB GPU) with `rep=10` iterations and 5 timing runs per configuration.

Table 1: CUDA dilation performance as a function of image size (`rep`=10, averaged over 5 runs).

Width	Height	Pixels	Avg time (s)	Throughput (Mpix/s)	BW (GB/s)
512	512	0.26 M	0.0155	169.54	0.34
1024	1024	1.05 M	0.0166	631.96	1.26
1344	1344	1.81 M	0.0218	828.76	1.66
2048	2048	4.19 M	0.0210	1998.95	4.00
4096	4096	16.78 M	0.0311	5390.07	10.78
8192	8192	67.11 M	0.0826	8121.06	16.24

3.1 Analysis

Several observations can be drawn from Table 1:

- **Throughput scales strongly with image size.** Moving from 512×512 to 8192×8192 increases throughput nearly 48× (from 170 to 8121 Mpix/s). Small images do not provide enough parallelism to saturate the 6912 CUDA cores of the A100; large images keep the GPU fully occupied.

- **Latency dominates for small images.** The 512×512 and 1024×1024 cases take almost the same wall-clock time ($\sim$15–17 ms) despite a $4\times$ difference in pixel count. This time is dominated by kernel launch overhead and PCIe transfer latency rather than computation.
- **Memory bandwidth grows but stays well below the A100 peak.** At 8192×8192 the effective bandwidth is 16 GB/s, far below the A100's peak HBM2e bandwidth of $\approx$2 TB/s. This is expected: the stencil computation is compute-bound (each pixel performs 121 conditional memory lookups), not bandwidth-bound.
- **Constant memory is beneficial.** The mask fits entirely in the 64 KB constant memory cache. All 256 threads in a block read the same mask element in the innermost loop, so the access is broadcast in a single transaction — equivalent to reading a single value.

4 Compilation and Execution

```
# CUDA version
module load cuda
nvcc -O2 cuda-dilate.cu -o cuda-dilate
./cuda-dilate 10 < random-input-4096.pgm > out.pgm

# OpenMP version
module load gcc
gcc -std=c99 -O2 -fopenmp omp-dilate.c -o omp-dilate
OMP_NUM_THREADS=8 ./omp-dilate 10 < random-input-4096.pgm > out.pgm
```

On Leonardo the SBATCH script `dilate-cuda-job.sh` compiles and runs all benchmarks automatically:

```
sbatch dilate-cuda-job.sh
```

5 Conclusion

The CUDA implementation parallelises the dilation operator effectively for large images: at 8192×8192 it achieves over 8 Gpixels/s. The key design choices — constant memory for the mask, ping-pong device buffers for multi-iteration runs, and a 2D thread decomposition matching the image geometry — keep the implementation simple, correct, and performant. The OpenMP version provides a straightforward shared-memory baseline requiring only a two-line change to the serial code.

Acknowledgements

The student used copilot to help generate implementation of the code and to write the report.